

ChatGPT ▾

✦ Upgrade

↗ Share



There is a dynamic iframe resizing problem with the web page at https://simonmajor.github.io/webgists/wiring_etc/index.html . The strategy is at an onload event to measure the iframe's `contentWindow.document.body.scrollHeight` and apply it as a height style property to the iframe. Various implementations have been tried, but for most the problem is the `scrollHeight` is incorrectly at a default value of 150px at the time of measurement.

Implementing it inline in the onload attribute of the iframe works correctly. This implementation is undesirable as it duplicates code when duplicates boilerplate code when used for multiple iframe per page and multiple pages.

Enumerating the iframes in javascript using `document.querySelectorAll` and attaching an onload event to the iframe element with `addEventListener` does not work correctly.

Enumerating the iframes in javascript using `document.querySelectorAll` and setting the onload attribute of the iframe element with does not work correctly. This is the currently active implementation.

Implementations of other implementations can be found in
the sources commented out.

I inspected your description, and the behavior you're seeing is consistent with a subtle timing difference between the iframe's native load event and when the browser has completed layout and calculated the document height.

The clue is:

Inline `onload= "..."` works.

`addEventListener('load', ...)` and dynamically assigning `iframe.onload = ...` do not.

Normally those should be equivalent, but in practice browsers don't always schedule them identically when dealing with nested documents, especially on GitHub Pages where rendering, fonts, stylesheets, and layout can complete after the initial iframe load event.

Why you get 150px

150px is the browser's default iframe height.

When you read:

`</>` JavaScript



```
iframe.contentWindow.document.body.scrollHeight
```

too early, the body has not yet been laid out, so the browser reports a height close to the default iframe size instead of the final content size.

The inline version works because it executes slightly later in the browser's event processing sequence than your dynamically-attached handlers.

Another issue: `body.scrollHeight` is often the wrong measurement

For iframe auto-sizing, the most reliable measurement is usually:

```
</> JavaScript   
  
Math.max(  
  doc.body.scrollHeight,  
  doc.documentElement.scrollHeight,  
  doc.body.offsetHeight,  
  doc.documentElement.offsetHeight  
)
```

Different browsers place the "true" height on different elements.

The robust solution

Instead of measuring immediately on `load`, defer measurement to the next rendering frame.

```
</> JavaScript 
```

```
function resizeIframe(iframe) {  
  const doc = iframe.contentDocument;  
  
  if (!doc) return;  
  
  requestAnimationFrame(() => {  
    const height = Math.max(  
      doc.body.scrollHeight,  
      doc.documentElement.scrollHeight,  
      doc.body.offsetHeight,  
      doc.documentElement.offsetHeight  
    );  
  
    iframe.style.height = `${height}px`;  
  });  
}
```

Then:

```
</> JavaScript  
  
document.querySelectorAll('iframe.auto-height')  
  .forEach(iframe => {  
    iframe.addEventListener('load', () => resizeIframe(iframe));  
  });
```

and

```
< HTML  
  
<iframe class="auto-height" src="..."></iframe>
```

Even more reliable

If the iframe content loads fonts or images after the load event, a single measurement can still be wrong.

In that case:

```
JavaScript

function resizeIframe(iframe) {
  const doc = iframe.contentDocument;

  if (!doc) return;

  const update = () => {
    iframe.style.height =
      Math.max(
        doc.body.scrollHeight,
        doc.documentElement.scrollHeight,
        doc.body.offsetHeight,
        doc.documentElement.offsetHeight
      ) + 'px';
  };

  requestAnimationFrame(update);

  setTimeout(update, 50);
  setTimeout(update, 250);
  setTimeout(update, 1000);
}
```

This catches late layout changes from fonts and images.

Best practice for many pages and many iframes

Since all your iframe content is same-origin (`simonmajor.github.io` loading other pages from the same site), the cleanest solution is to let the iframe tell the parent its size.

Parent page

</> JavaScript



```
window.addEventListener('message', event => {

    if (event.data.type !== 'iframeHeight')
        return;

    const iframe = document.getElementById(event.data.id);

    if (iframe)
        iframe.style.height = `${event.data.height}px`;

});
```

Parent iframe markup

HTML



```
<iframe
  id="wiring1"
  src="some-page.html">
</iframe>
```

Inside every iframe page

</> JavaScript



```
function reportHeight() {

    const height = Math.max(
        document.body.scrollHeight,
        document.documentElement.scrollHeight
    );

    parent.postMessage({
        type: 'iframeHeight',
        id: 'wiring1',
        height: height
    }, '*');
}

window.addEventListener('load', reportHeight);
window.addEventListener('resize', reportHeight);
```

This approach is extremely reliable because the measurement occurs inside the document being measured.

What I would do for your GitHub Pages site

Looking at your architecture (multiple pages containing multiple reusable iframe fragments), I'd use:

1. A shared `iframe-resize.js` .
2. Attach listeners with `addEventListener('load', ...)` .
3. Measure inside `requestAnimationFrame` .
4. Use a `ResizeObserver` within the iframe document when available.

Example:

```
</> JavaScript   
  
document.querySelectorAll('iframe.auto-height').forEach(iframe => {  
  
    iframe.addEventListener('load', () => {  
  
        const doc = iframe.contentDocument;  
  
        const update = () => {  
            iframe.style.height =  
                Math.max(  
                    doc.body.scrollHeight,  
                    doc.documentElement.scrollHeight  
                ) + 'px';  
        };  
  
        update();  
  
        new ResizeObserver(update)  
            .observe(doc.body);  
    });  
});
```

For same-origin pages, this is generally the most scalable and maintainable solution. It avoids inline handlers entirely, works for any number of iframes, and automatically adapts if the iframe content changes size after loading.

If you can point me at the actual JavaScript file currently used on that page (or paste the relevant code), I can explain precisely why the current `querySelectorAll(...).forEach(...)` implementation fires too early while the inline

onload version succeeds.



The JavaScript file currently used on the page is https://simonmajor.github.io/webgists/wiring_etc/includes/files.js

I tried to retrieve `files.js` , but the web search system can't access raw GitHub Pages JavaScript files directly and instead returned unrelated results. So I can't yet inspect the actual code. [Gist +1](#)

To determine exactly why your current implementation behaves differently, I need the contents of `files.js` (or at least the iframe-related portions).

Could you either:

- paste the contents of `files.js` , or
- provide a raw URL that serves the JavaScript source directly (for example a GitHub repository URL rather than the GitHub Pages URL)?

In particular I'm looking for:

 JavaScript

```
document.querySelector(...)
```



the code that assigns:

```
</> JavaScript
```



```
iframe.onload = ...
```

or

```
</> JavaScript
```



```
iframe.addEventListener('load', ...)
```

and the function that reads:

```
</> JavaScript
```



```
iframe.contentWindow.document.body.scrollHeight
```

Once I can see the actual implementation, I can explain precisely why the inline `onload` succeeds while the delegated versions fail, rather than just describing the general browser timing issues.

     ...  Sources

The content of that file are as follows:

```
/* function adjustIframeHeight(felem) {  
    console.log("adjustIframeHeight entry");  
    // console.log("Type of felem:", typeof felem, "Value:",  
felem);  
    try {  
        const diframeContent = felem.contentDocument ||  
felem.contentWindow.document;  
        // console.log("Type of diframeContent:", typeof
```

Show more ▾

The bug is actually visible in the active implementation.

You currently have:

JavaScript



```
document.querySelectorAll("iframe.full").forEach((iifr) => {  
    iifr.onload = (function (o) {  
        o.style.height = o.contentWindow.document.body.scrollHeight + "px";  
    })(iifr);  
});
```

What's happening

The expression:

```
</> JavaScript   
  
(function (o) {  
    o.style.height = o.contentWindow.document.body.scrollHeight + "px";  
})(iifr);
```

is an **immediately invoked function expression** (IIFE).

It runs **right now**, while the page is being parsed.

The result of that expression is then assigned to `onload` .

Since the function has no `return` , the assignment becomes:

```
</> JavaScript   
  
iifr.onload = undefined;
```

So no load handler is actually attached.

In other words:

```
</> JavaScript   
  
iifr.onload = someFunction();
```

calls the function immediately,

whereas

JavaScript

```
iifr.onload = someFunction;
```

attaches the function for later execution.

Why the inline version works

The inline version:

HTML

```
<iframe
  onload='javascript:(function(o){
    o.style.height=o.contentWindow.document.body.scrollHeight+"px";
  })(this));'>
</iframe>
```

is evaluated when the iframe's load event fires.

That means:

JavaScript

```
o.contentWindow.document.body.scrollHeight
```

is measured after the iframe has loaded.

The active JavaScript implementation measures it before the iframe loads.

That alone explains the behavior you're seeing.

The equivalent JavaScript

The true equivalent of your working inline code is:

⌵ JavaScript



```
document.querySelectorAll("iframe.full").forEach((iframe) => {  
  iframe.onload = function () {  
    iframe.style.height =  
      iframe.contentWindow.document.body.scrollHeight + "px";  
  };  
});
```

or:

⌵ JavaScript



```
document.querySelectorAll("iframe.full").forEach((iframe) => {  
  iframe.onload = () => {  
    iframe.style.height =  
      iframe.contentWindow.document.body.scrollHeight + "px";  
  };  
});
```

Why your earlier addEventListener attempts probably failed

I noticed another issue in the commented code:

```
</> JavaScript   
  
fbody.addEventListener(  
    evt,  
    adjustIframeHeight(iifr),  
    false  
);
```

Again, you're calling the function immediately.

That becomes:

```
</> JavaScript   
  
fbody.addEventListener(  
    evt,  
    undefined,  
    false  
);
```

because `adjustIframeHeight()` returns nothing.

The correct form would be:

```
</> JavaScript 
```

```
fbody.addEventListener(  
  evt,  
  () => adjustIframeHeight(iifr),  
  false  
);
```

or

```
 JavaScript   
  
fbody.addEventListener(  
  evt,  
  adjustIframeHeight.bind(null, iifr),  
  false  
);
```

The same mistake appears multiple times in the commented-out experiments.

Another timing issue

Even after fixing the handler attachment, there is a second problem.

GitHub Pages serves HTML that may contain:

- external CSS
- web fonts
- images
- syntax highlighting

The iframe's `load` event can occur before final layout settles.

So:

```
</> JavaScript  
  
body.scrollHeight
```



may still be 150 or some intermediate value.

A robust implementation

I'd write:

```
</> JavaScript
```



```
function resizeIframe(iframe) {  
  
  const doc =  
    iframe.contentDocument ||  
    iframe.contentWindow.document;  
  
  const height = Math.max(  
    doc.body.scrollHeight,  
    doc.documentElement.scrollHeight,  
    doc.body.offsetHeight,  
    doc.documentElement.offsetHeight  
  );  
  
  iframe.style.height = `${height}px`;  
}  
  
document.querySelectorAll("iframe.full").forEach((iframe) => {  
  
  iframe.addEventListener("load", () => {  
  
    requestAnimationFrame(() => {  
      resizeIframe(iframe);  
    });  
  
    setTimeout(() => resizeIframe(iframe), 100);  
  });  
});
```

This:

1. scales to any number of iframes,
2. avoids inline JavaScript,
3. waits for the iframe load event,
4. waits one render cycle,

5. performs a second measurement after late layout changes.

The best solution for your site

Because all your iframe content appears to be same-origin (`*.github.io/webgists/...`), I'd go one step further and use a `ResizeObserver` .

⌵ JavaScript



```
function attachAutoResize(iframe) {

  iframe.addEventListener("load", () => {

    const doc =
      iframe.contentDocument ||
      iframe.contentWindow.document;

    const update = () => {

      iframe.style.height =
        Math.max(
          doc.body.scrollHeight,
          doc.documentElement.scrollHeight
        ) + "px";
    };

    update();

    new ResizeObserver(update)
      .observe(doc.body);
  });
}

document
  .querySelectorAll("iframe.full")
  .forEach(attachAutoResize);
```

This automatically resizes whenever the iframe content changes and eliminates the need for inline `onload` handlers entirely.

The most important discovery from your source, however, is that the current

implementation is not actually registering a load handler at all—it is executing the `resize` immediately and assigning `undefined` to `iframe.onload`. That explains why it behaves differently from the inline version.

